

Design Architecture Review (DAR) – Final Project Template

Project Information

****Project Title:** ** GameCast Director

****Team Members (names + uNIDs):** ** Noah Hart(u1537288), Divith Nagaraj Reddy(u1482564) and Adhithya Narayanan(u1475663)

1. Executive Summary

In multiplayer games, it is important to have various perspectives displayed to players or spectators. With the help of **GameCast director plugin**, the process would be streamlined to ensure that moments in multiplayer games are not missed.

The plugin will act as a director, providing users with the ability to **switch between different players and regions effortlessly**. The ability to also move around freely in space will be useful to provide cinematic shots as well as capture the action moments.

We intend the plugin to be used by cinematic/narrative designers for collecting action moments from live gameplay and Esports organizers for providing the spectators with complete experience of a match.

1.1. Action Moments

Action moment is defined as a significant gameplay event that impacts the state of players and the game itself. Moments such as engaging in combat with other players, successfully eliminating said enemies, activating special abilities and even narrative triggers are some of the events that fit into the category. Based on these events, they are further categorized into First Person, Third Person, and Spectator mode.

1.1.1 Player Perspective

Player Perspective mode is designed to have the most player immersion, so to keep that we provide the ability to have the camera bound to the player's perspective. Games that were used as reference were Counter Strike Global Offensive, whereby eliminations and clutch moments are captured, Valorant emphasizes ability-driven engagements as the moments that need to be captured.

[Counter Strike Global Offensive Clips](#)[Valorant Masters Clips](#)

1.1.2 Third Person

Third Person mode provides a clear view of the player's actions whether it be in combat or traversing the environment. Combat actions are classified as dodge, enemy takedowns, etc., and traversal actions are classified as sprinting, vaulting or environmental interactions. The aim is to have the camera be placed at the right offset for complete visibility of these actions with certain features like distance-based zoom, target weighted framing with smooth interpolation. Games used for reference were Fortnite, large scale multiplayer with traversal and combat, Marvel Rivals, team combat with ability usage needing constant reframing.

[Marvel Rivals](#)

1.1.3 Spectator

Spectator mode is more of a defined set of points in the game where the cameras will be placed with a focus on broadcast-oriented viewpoints. Moments of team rotations, objective captures and multi team engagements are a few of the instances whereby the camera would dynamically transition to one of the predefined positions, with it being placed at the appropriate field of view of the area. Game references are Overwatch 2, where tactical views are employed to provide the optimal viewing experience.

[Overwatch Spectator Mode](#)

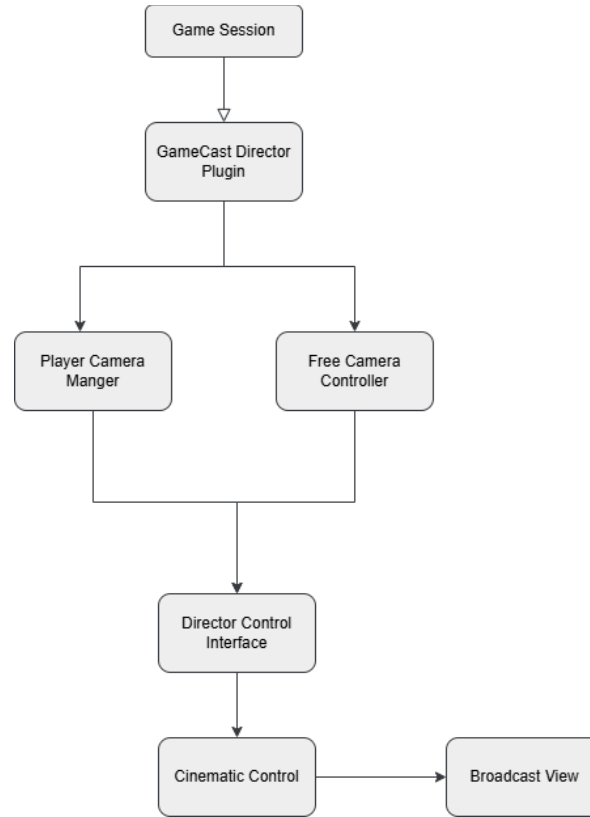
1.1.4 Free Roam

Free Roam mode provides a detached ability to move camera freely in the environment with no clipping. With aerial captures of environmental destruction or AOE (Area of Effect) abilities, are some of the moments that can be captured. The camera will be controlled independently by organizers or broadcasters, which is mostly inspired by fly-through cameras in Fortnite, and certain free camera captures in Marvel Rivals.

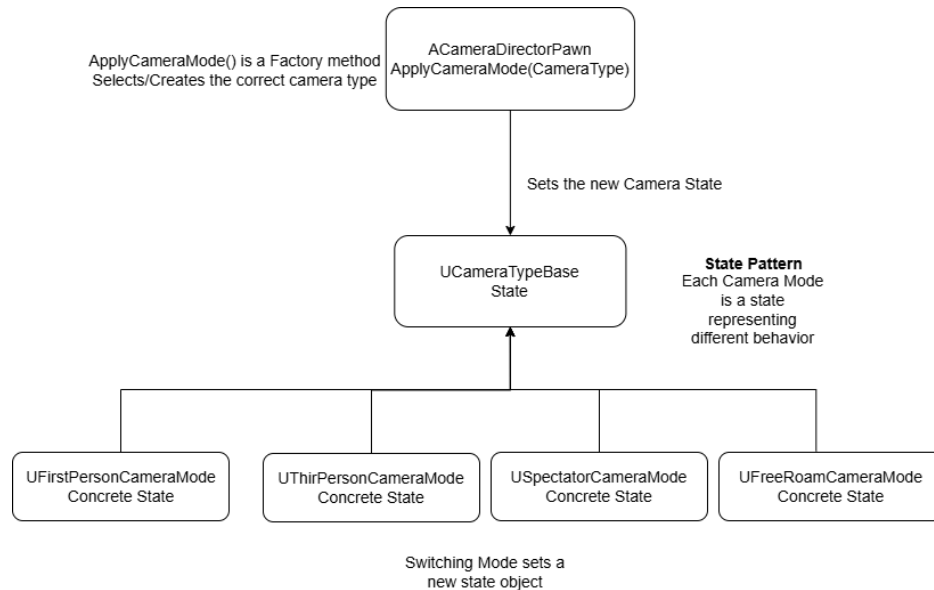
[Fortnite Free Roam Drone](#)

2. High-Level Architecture

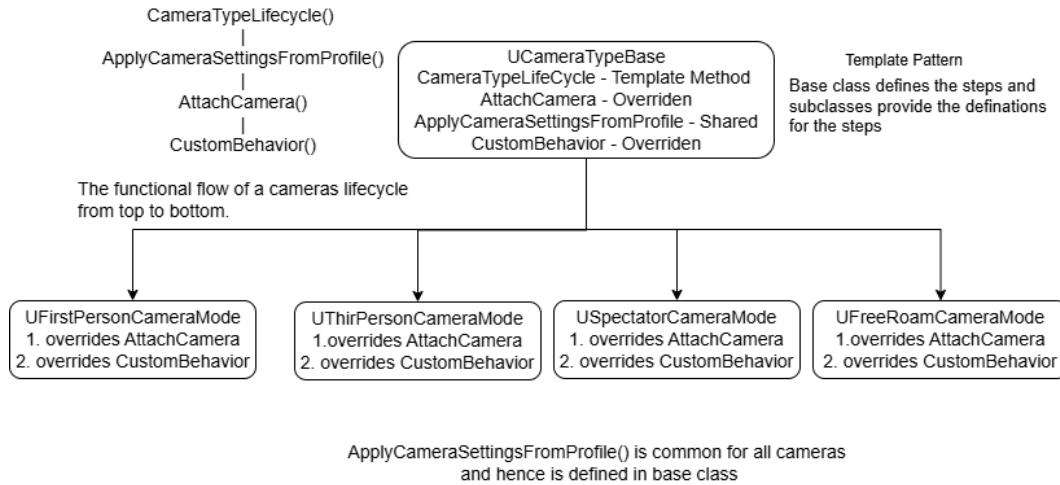
- Key Modules/Classes – list and describe
- Data Flow – explain how data/events move



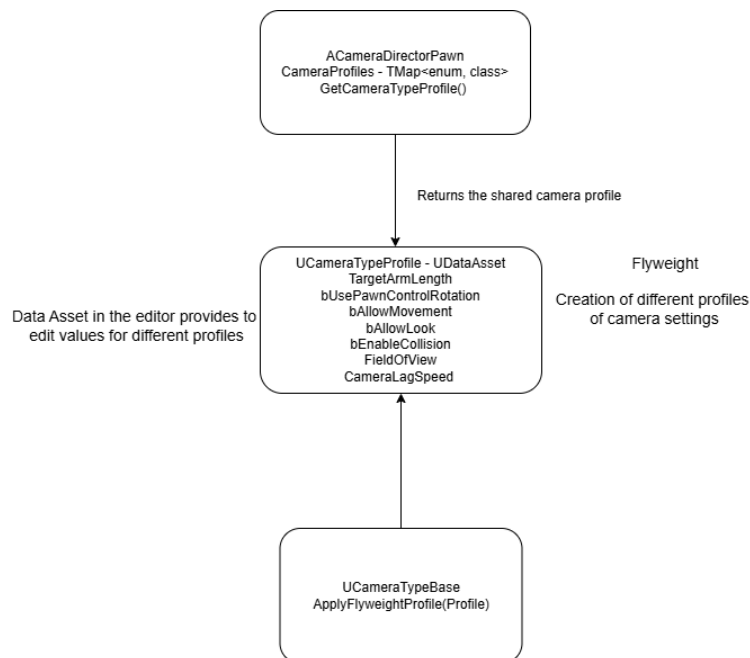
System Overview

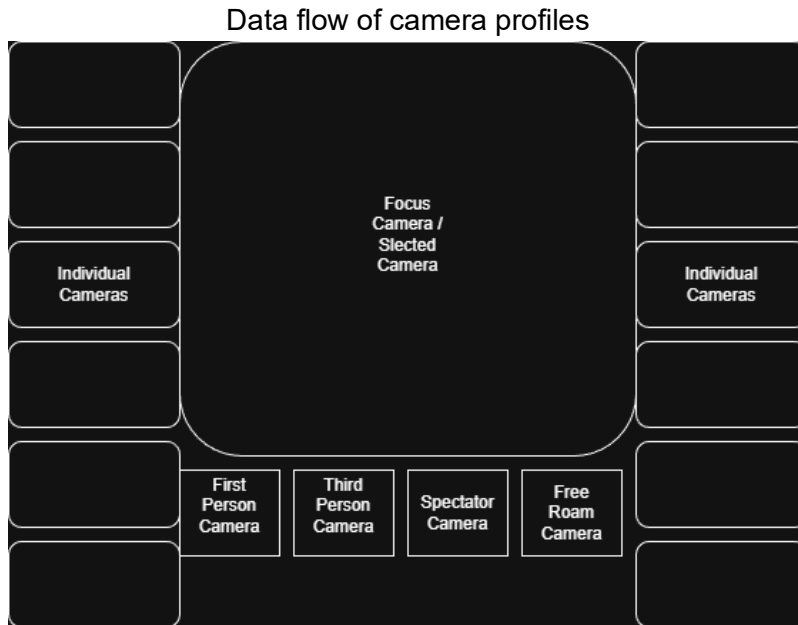


CameraType runtime state switch functionality



CameraTypeBase Class Diagram and programming pattern





Final Design Layout of UI

3. Design Decisions & Patterns

Document ≥ 10 Game Programming Patterns (≥ 1 per type).

For each pattern, describe:

- Where it's used
- Why it's used (problem it solves)

1. State Pattern

- The method of changing the typing and behavior of the object at runtime instead of using condition statements.
- Used for switching between Camera Modes and each mode has its own class and set of behavior.
- Provides the ease of extensibility of the Camera modes and is easily testable for multiple camera modes.

2. Open-Close Principal

- The CameraModes class is closed for modification and easily extendable to add new modes.
- Each new camera mode must be inherited from the CameraModeBase class and have an implementation of AttachCamera and CustomBehavior method.
- Enables adding new code with minimal risk and the need to change legacy CameraMode code or the game code to fetch the new modes.

3. Command Pattern

- An action is combined into a type of object which can be stored, queued, or executed later instead of binding logic directly to the action.
- Enables the support of actions such as undo/redo, recording and playback with each becoming a command object.
- It ensures that we can easily alter inputs without changing gameplay as well as making gameplay modular without cluttered input logic code.

4. Observer Pattern

- Triggers behavior on the listeners only when a change occurs in the subject.
- The camera switches to the player by having the camera listen to the events of the player and trigger behavior accordingly.
- Reduces polling logic for the camera, which in turn helps in decoupling the change and effect with no spaghetti code.

5. Template Pattern

- Creates the basis of the procedure in the base class allowing subclasses to fill in the functionalities.
- CamerModeBase is an abstract base class which enforces the defined structure of the subclasses providing a shared logic.
- Ensures that the camera lifecycle is consistent with each existing mode as well as giving the structure for future modes.

6. Factory Pattern

- Creates objects of a class without exposing the type of the class which is handled at runtime.
- CameraDirector calls the CameraTypeLifecycle method by creating an instance of CameraModeBase instead of creating objects of specific subclasses.
- Ensures the right subclass methods are called automatically with minimal code instead of individually calling the specific class functions.

7. Flyweight

- Sharing identical data amongst different objects by making the data intrinsic.
- Creation of different profiles to handle per instance and shared data across different cameras.
- Improves performance and ease of access to edit the variables at a singular point.

8. Single Responsibility Principal

- A class should have only one reason to change.

- Each class like CameraDirector is used only for orchestration, CameraMode is used only for camera behaviors etc.
- Easier for identifying bugs of specific features, and the ability to reuse certain components like configuration profiles for the cameras.

9. Liskov Substitution Principle

- Subclass should be usable through its base class without breaking the system.
- All camera modes override the same methods from CameraModeBase without requiring any additional logic.
- Ensures safe and effective polymorphism without causing any bugs when switching between objects.

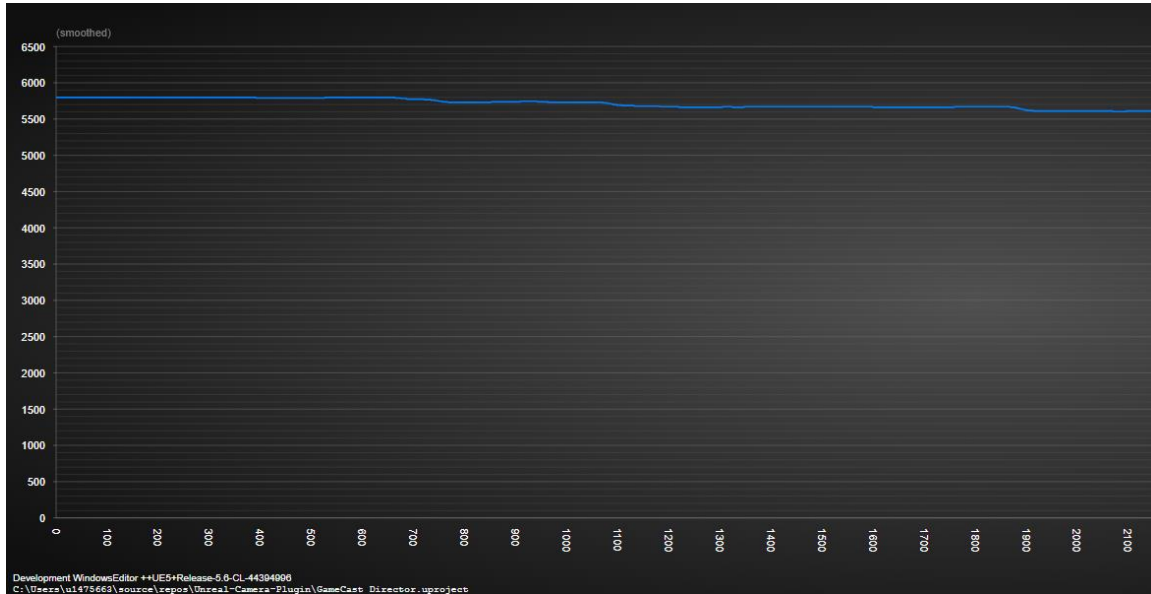
10. Dependency Inversion Principle

- The use of abstract classes instead of relying on concrete classes for implementation.
- InputCommands and the CameraModeBase are what the CameraDirector depends on which are abstracted.
- Avoid hard code dependency on particular camera modes or input bindings. High level modules do not depend on low level modules.

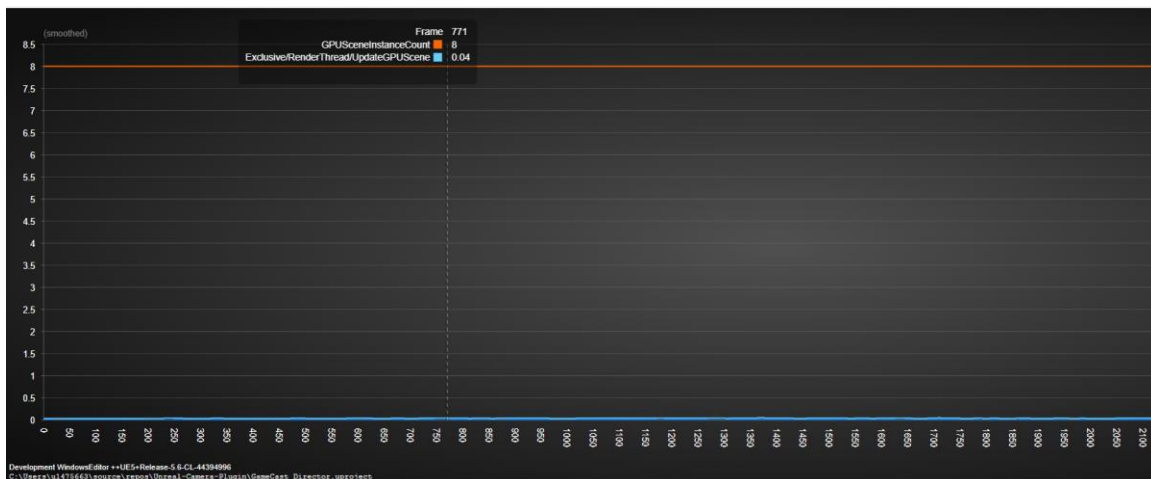
4. Unreal-Specific Integration

- Editor Extension – what part of the Editor is extended?
- Custom Profiling Stats – What Do You Track?
- Drop-in Capability – how to integrate in any UE project?

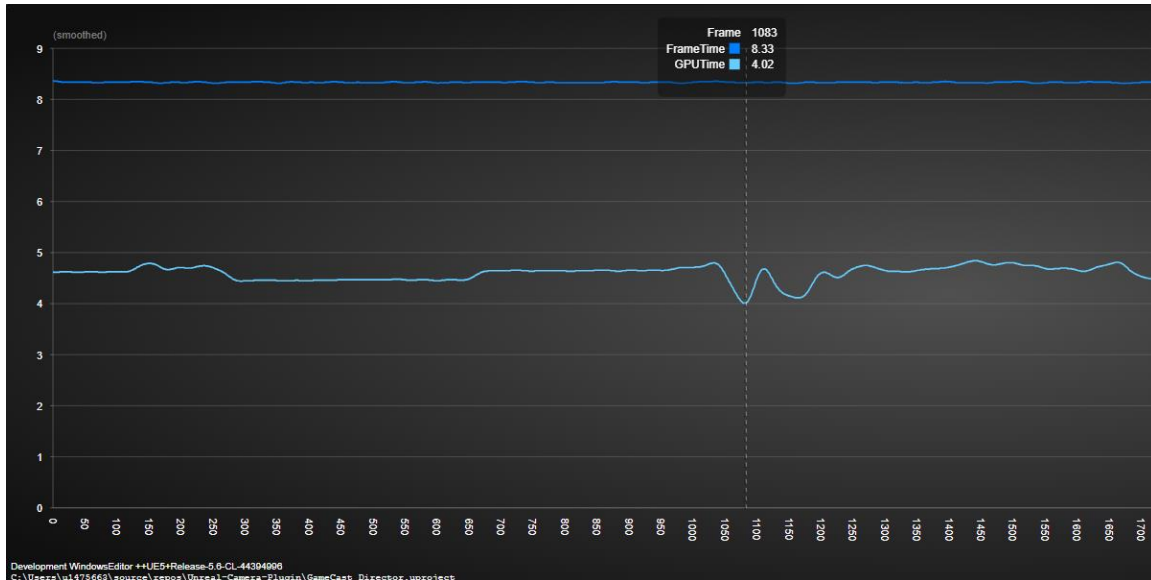
Currently we are tracking the RAM used by the program when it is running as standalone. This would be to track the memory taken up with by the saved recordings



We will also be tracking the GPU instances that are being used while running it in standalone. Given the additional rendering in a UI, we would monitor spikes in the graph and logs “GPU-Scene Instance Data overflow detected”.



Finally, to ensure smooth frames and transitions, we are tracking the FrameTime and GPUPTime when the program is running as standalone.



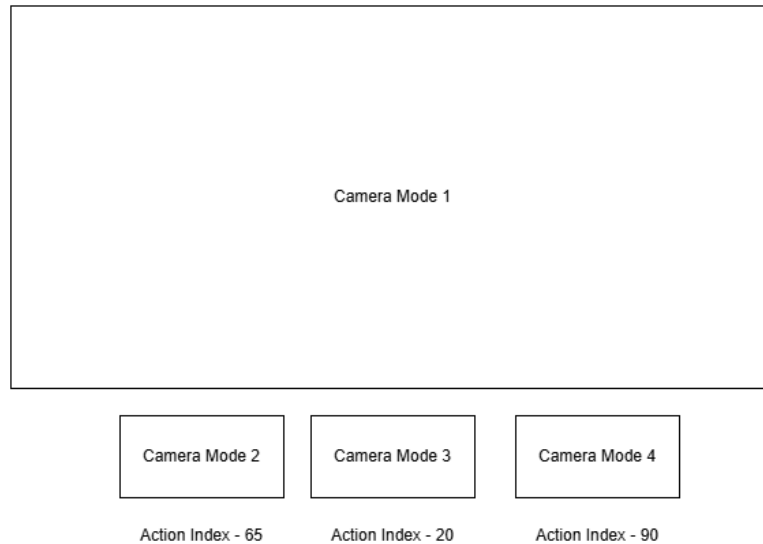
We will be adding more profiling checks by next week once we have the features in place with a more in-depth look at the performance of the features with a comparison between initial stats and after optimization stats.

5. Alternatives Considered

List 1–2 alternative designs considered and why they were rejected.

1. The initial design was to include the features of camera modes in a network setting. Given the scope and time remaining, we have decided to focus on creating different modes and the ability to capture action moments with an emphasis on optimizing the process.
2. The idea was provided to have a rotating or floating screen setting wherein there would be different modes and based on a variable value called Action Index the screen would become the main screen while the other one would be resized to fit

in with the other smaller screens.



3. The other design idea was to have the screen divided into two screens and showcase both the teams with the option to switch modes for that specific team. Most of the games are team-based games with 5-6 players on each team, so to fit this we went ahead with this design idea.

The pictorial representation is still being discussed and worked on to find the ideal fit and placements of the various modes in the interface.

6. Risks & Mitigations

- Technical Risks (e.g., performance, compatibility)
- Scope Risks (e.g., ambition too high)
- Mitigation strategies

1. Intuitive

- The target audience of the plugin would be ESports organizers, and we would have to ensure that the options provided in the user interface are easily understandable and provide the end user with the easiest way to navigate and to switch between modes.
- To ensure that we don't stray away too much from the norm, we plan on discussing with our Universities ESports manager and getting their opinions on what they would like to control and capture during a match.

2. Performance

- The use of a User Interface will add a performance overhead on the GPU if we use Unreal Motion Graphics as we plan on providing view for multiple modes on the UI.
- The solution we are looking to implement is the use of Windows Forms so that we can get more flexibility and better performance as it won't have the overhead of Unreal Motion Graphics.

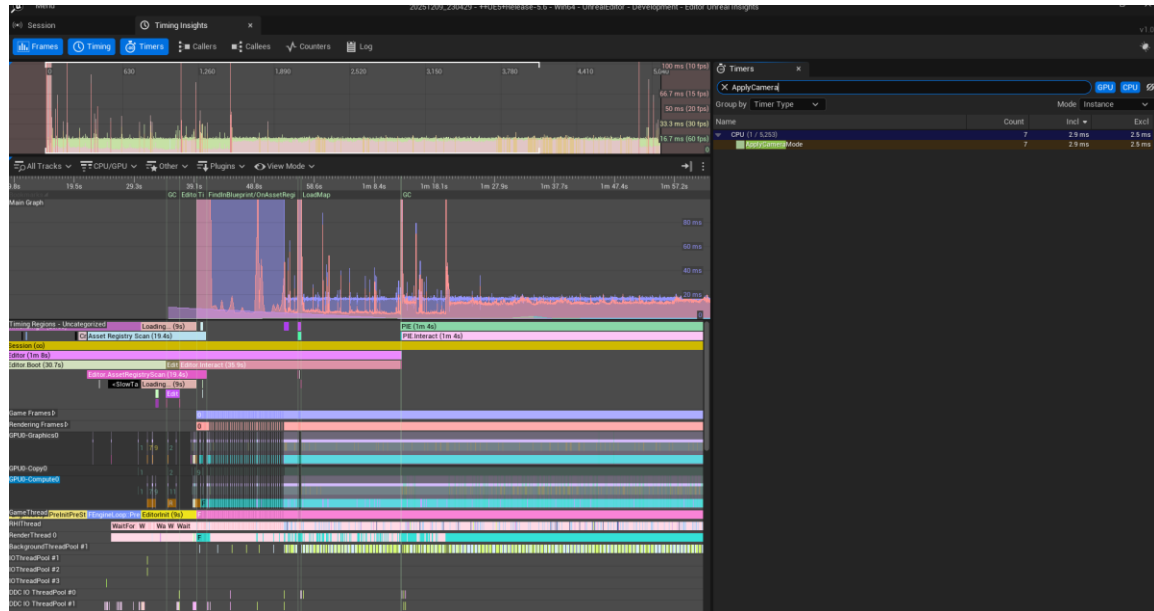
3. Recording Action Moments

- The design of recording an action moment and storing it would cause a significant performance hit on the system on which it would be running.
- We plan to have profiling and set a benchmark from the get-go to ensure there are no spikes in performance and try to find the optimal data structure to store the recording and clean the unused.

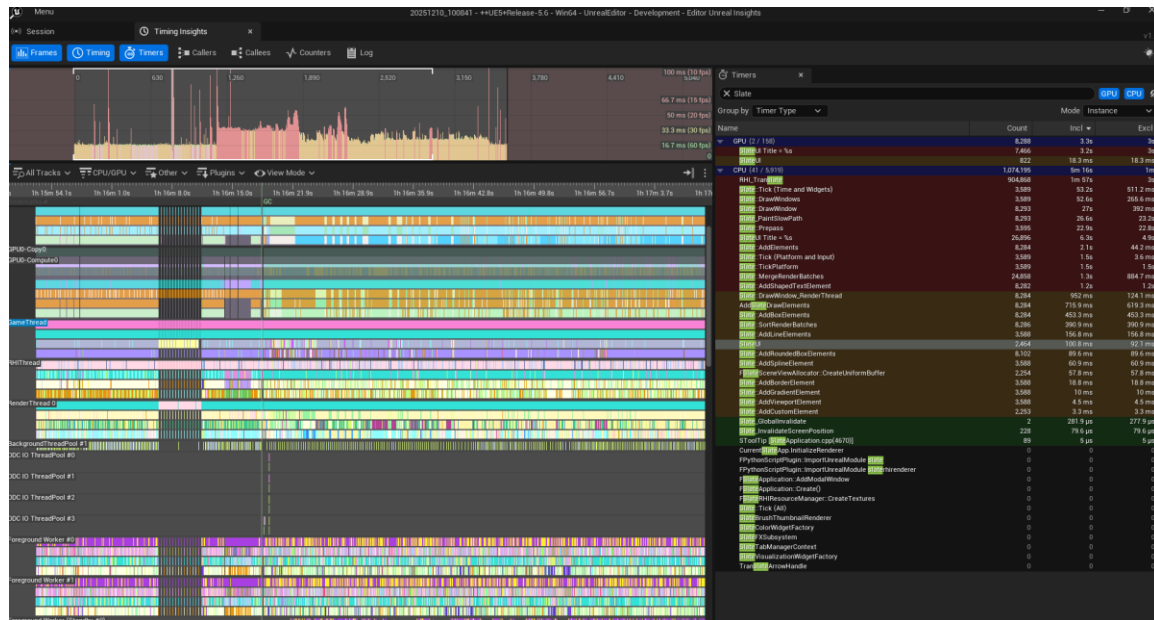
7. Testing & Validation Plan

- How will you prove the plugin works?
- Tests, performance checks, scenarios
- How profiling stats will be validated

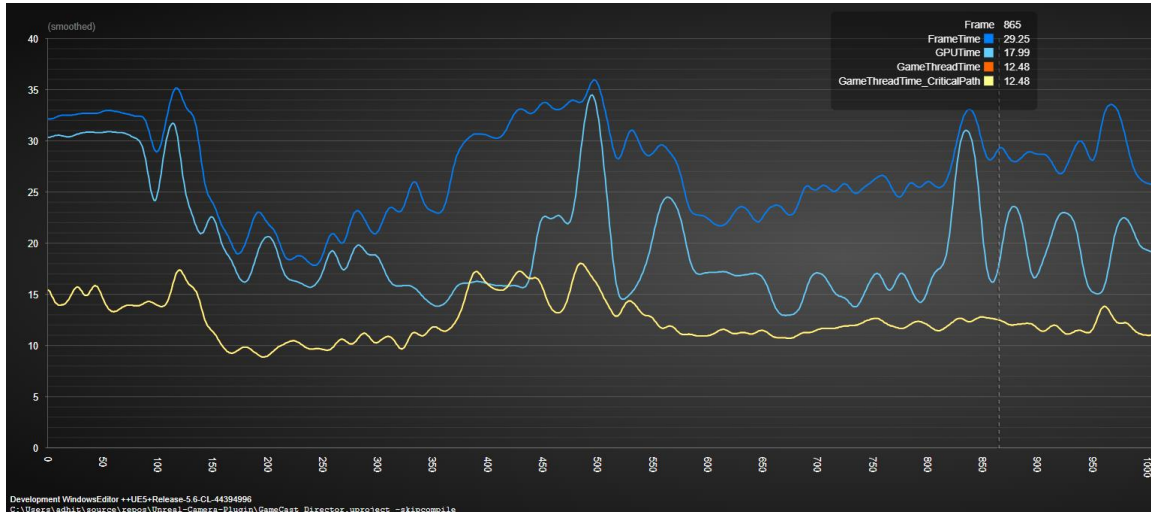
The plan for testing would be to have the commands for profiling be setup in a batch script with a Windows Task Scheduler running it every day at 17:00 MST and have the SVG uploaded to a drive giving time for our team to review the performance of the plugin after certain features are implemented.



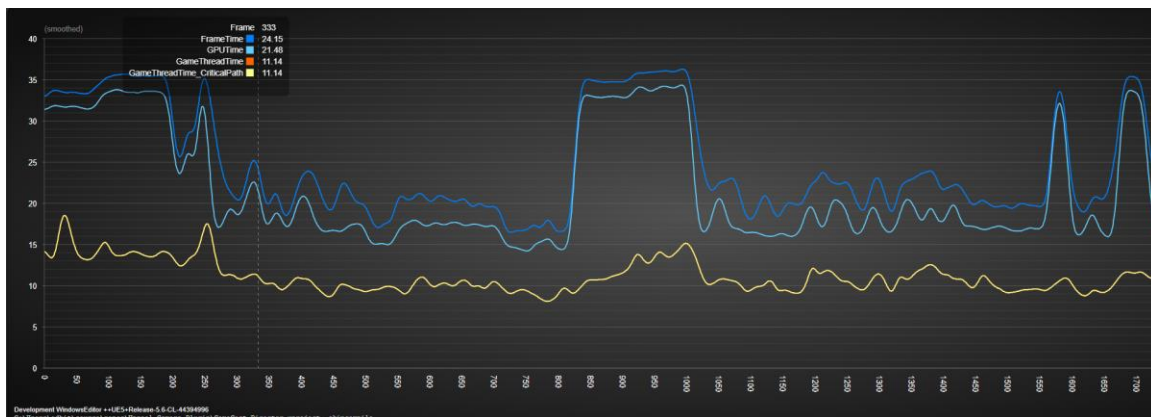
The ApplyCameraMode is the heart of the plugin, and it takes only <0.35ms to execute. It's called only when the mode of changes and doesn't affect the runtime efficiency.



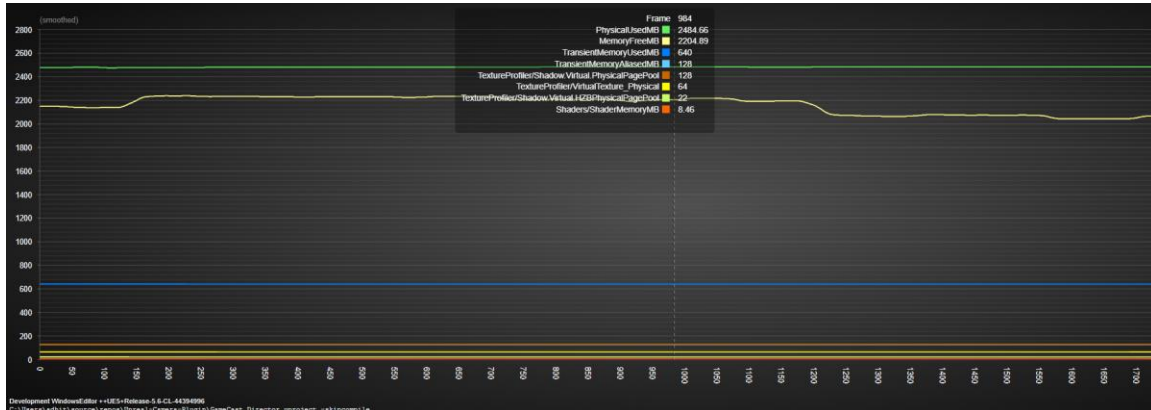
We are currently polling for changes for the UI to render the camera's perspective. With the addition of delegates, we would have significant lesser redraws of the UI providing improved performance of the plugin. The optimization is planned to be added for future iterations.



*The custom profiling is added to CameraRecordMoments to track the CPU and GPU performance during the method calls. The rendering operation takes up most of the performance on the GPU, and there are not any spikes on the CPU.



*Similar custom profiling was added to CameraDirector especially for the Tick and the camera switch method. Transitions cause spikes in the GPU needing to redraw certain elements, but the CPU remains consistent without any spikes.



*The profiling is used for tracking the memory for CameraDirectorPawn. There is stable runtime memory without any leaks or spikes during transitions. The plugin doesn't introduce any additional rendering or streaming related allocations.

*All the profiling were noted from the PIE.

8. Deliverables Alignment

Checklist:

- ☐ DAR Document
- ☐ C++ Plugin (drop-in)
- ☐ Profiling Stats + Editor Extension
- ☐ ≥10 Patterns documented
- ☐ User Manual
- ☐ (Optional) GitLab CI/CD
- ☐ Marketplace Package
- ☐ Live Demo Plan

9. Appendix

- Glossary
- References

Why DAR is Different from a Technical Design Document

A Technical Design Document (TDD) focuses on class-level details and low-level implementation.

A Design Architecture Review (DAR) focuses on system-level thinking and trade-offs:

- Emphasizes WHY over HOW
- Aligns with industry architecture reviews

Page no. 15

- Prepares teams to defend design choices
- Ensures scalability, maintainability, and clarity

Scope Note

Keep DAR ~10–12 pages max (including diagrams). Clear, professional, and concise.